



CBU5000V210 UWB SoC

Flash Encryption Guide V1.1

ShenZhen ChipsBank Technology Co., Ltd.

<http://www.chipsbank.com/>

7th Floor, Building 12, Phase II, Shenzhen Software Park,

Kejizhong 2nd Road, Nanshan District

Table of Content

1 INTRODUCTION	3
1.1 PURPOSE	3
1.2 SCOPE	3
2 HARDWARE OVERVIEW	3
2.1 KEY FEATURES	3
3 THEORY OF OPERATION	4
3.1 ARCHITECTURE	4
3.2 OPERATING MODES	4
4 REGISTER CONFIGURATION	5
4.1 INITIALIZATION	5
5 APPLICATION EXAMPLES	6
5.1 EXAMPLE 1	6
5.2 EXAMPLE 2	6
5.3 EXAMPLE 3	8
6 FIRMWARE APIS	9
7 NOTE	9
8 REVISION HISTORY	10
9 DISCLAIMERS	11

List of Figures

Figure 3-1 FW security model	4
Figure 4-1 AES key and seed configuration code	5
Figure 4-2 Serial port operation log	5
Figure 5-1 FLM project configuration interface	6
Figure 5-2 Modification content of auto.bat batch script for plaintext bin generation	7
Figure 5-3 Keil After-build run settings	7
Figure 5-4 Key and Seed mapping table	8
Figure 5-5 key.key file format example	8
Figure 5-6 Offline encryption tool operation method	8

1 Introduction

1.1 Purpose

This application note aims to detail the complete operation process of the chip Flash AES hardware encryption function, including permanent full-Flash AES encryption enabling based on eFuse and selective partial Flash AES encryption based on register configuration. It provides standardized and minimal practical operation steps for engineers to realize firmware encryption, Flash data automatic encryption and decryption, and solve the problems of plaintext firmware startup failure and Flash data security protection.

1.2 Scope

Covered Features: eFuse one-time burning enabling of permanent full-Flash AES automatic encryption and decryption, encrypted firmware downloading based on Keil+FLM file and SF100 tool, register-based selective partial Flash AES encryption and decryption, and encryption effect verification methods.

Non-covered Features: AES algorithm principle deep analysis, other peripheral encryption functions, third-party non-matching download tool adaptation, and advanced firmware encryption customization development.

2 Hardware Overview

2.1 Key Features

1. Dual AES encryption modes: support eFuse permanent full-Flash automatic encryption and decryption, and register-based selective partial-Flash encryption and decryption, meeting different scene security requirements.
2. One-time irreversible eFuse burning: After AES function is enabled by eFuse burning, the chip only supports encrypted firmware operation, and plaintext firmware cannot start normally.
3. Automatic hardware encryption and decryption: The Flash controller independently completes data encryption during writing and decryption during reading without additional firmware algorithm calculation.
4. Multiple encrypted firmware downloading solutions: support Keil matched dedicated FLM file direct downloading and SF100 tool encrypted bin file downloading.
5. Simple and verifiable encryption effect: Support register reading verification and Flash data reading check verification to confirm encryption and decryption validity.

3 Theory of Operation

3.1 Architecture

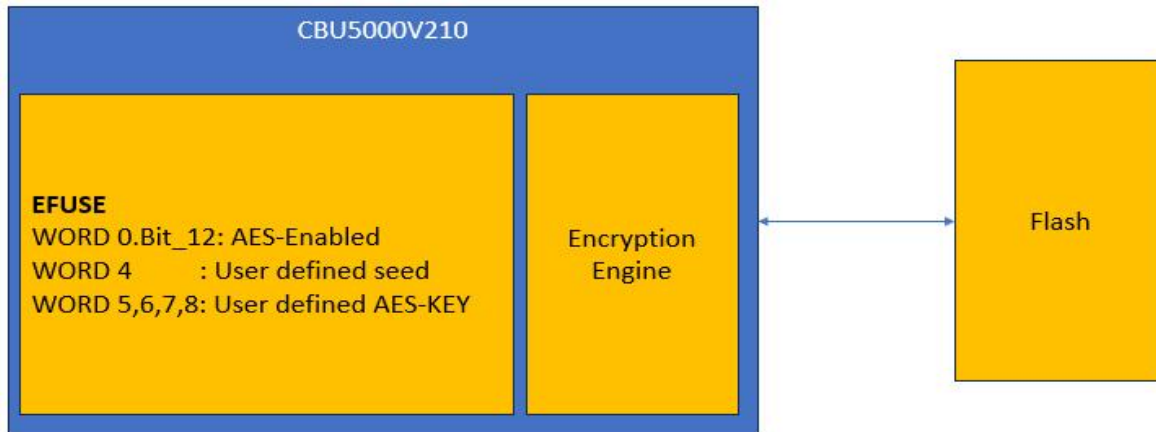


Figure 3-3: FW security model

3.2 Operating Modes

The Flash AES hardware encryption supports two mutually independent operating modes, which are applicable to different application scenarios:

Scenario 1: FW and user data encryption and decryption.

This mode is implemented by burning eFuse, which is irreversible. After enabling, the chip's entire Flash read-write process will automatically enable AES encryption and decryption. The chip can only run encrypted firmware, and any plaintext firmware cannot be identified and started. It is applicable to mass production and fixed equipment scenarios with high security requirements.

Scenario 2: User data encryption and decryption.

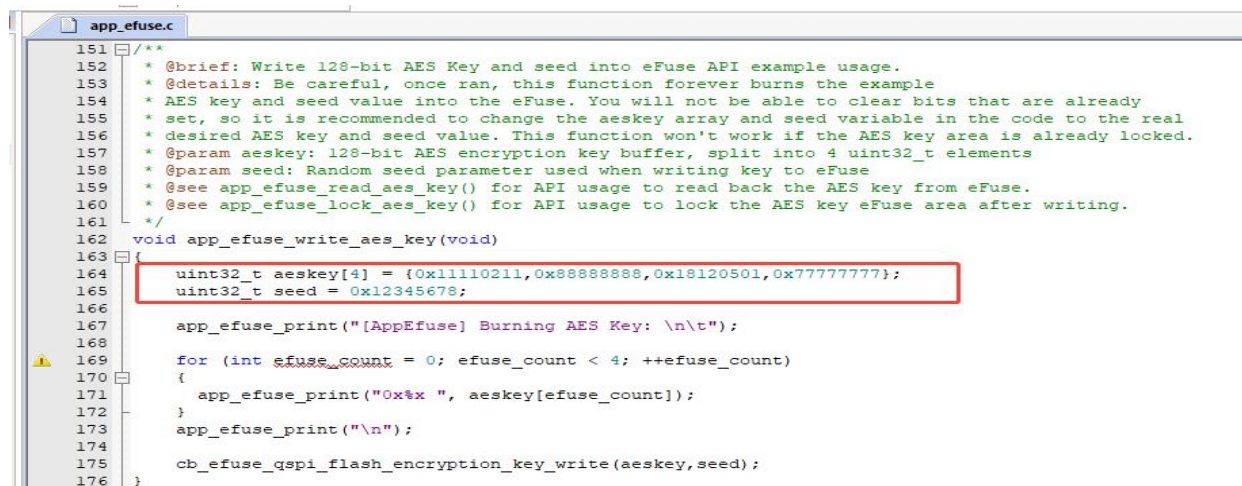
This mode is controlled by Flash master control register bits. By configuring the AES enable bit (1 for enable, 0 for disable) in the register, selective encryption of partial Flash write data can be realized. The function is flexible and reversible, and will not affect the startup of the overall firmware. It is applicable to scenarios where only key data in Flash needs encrypted protection and the firmware body does not need encryption.

4 Register Configuration

4.1 Initialization

Enable AES and Configure AES key in eFuse

1. Modify the 128-bit AES key and seed value in the `peripheral\_efuse` example of the SDK to customize your encryption configuration.



```

151 /**
152  * @brief: Write 128-bit AES Key and seed into eFuse API example usage.
153  * @details: Be careful, once ran, this function forever burns the example
154  * AES key and seed value into the eFuse. You will not be able to clear bits that are already
155  * set, so it is recommended to change the aeskey array and seed variable in the code to the real
156  * desired AES key and seed value. This function won't work if the AES key area is already locked.
157  * @param aeskey: 128-bit AES encryption key buffer, split into 4 uint32_t elements
158  * @param seed: Random seed parameter used when writing key to eFuse
159  * @see app_efuse_read_aes_key() for API usage to read back the AES key from eFuse.
160  * @see app_efuse_lock_aes_key() for API usage to lock the AES key eFuse area after writing.
161  */
162 void app_efuse_write_aes_key(void)
163 {
164     uint32_t aeskey[4] = {0x11110211, 0x88888888, 0x18120501, 0x77777777};
165     uint32_t seed = 0x12345678;
166
167     app_efuse_print("[AppEfuse] Burning AES Key: \n\t");
168
169     for (int efuse_count = 0; efuse_count < 4; ++efuse_count)
170     {
171         app_efuse_print("0x%x ", aeskey[efuse_count]);
172     }
173     app_efuse_print("\n");
174
175     cb_efuse_qspi_flash_encryption_key_write(aeskey, seed);
176 }

```

Figure 4-1: AES key and seed configuration code

2. Compile and download the example program, power on the device and prepare for serial port operation



```

[18:07:44.968]收←◆
Power-on reset
[AppEfuse] Received request to run to: 0
[AppEfuse] By default, tc are numbered as:
[AppEfuse] 0: (SAFE) Check if AES mode is enabled
[AppEfuse] 1: (SAFE) Read AES key
[AppEfuse] 2: (SAFE) Read User Config 2
[AppEfuse] 3: (SAFE) Read Chip ID
[AppEfuse] 4: ***DANGEROUS*** Enable AES mode
[AppEfuse] 5: ***DANGEROUS*** Burns AES key into eFuse
[AppEfuse] 6: ***DANGEROUS*** Set User Config 2
[AppEfuse] 7: ***DANGEROUS*** Lock AES Settings
[AppEfuse] 8: ***DANGEROUS*** Lock AES key from R/W
[AppEfuse] 9: ***DANGEROUS*** Lock User Config 2

[AppEfuse] Aes is enabled: 0 - FALSE

Please Send CCCC+command(0x00-0x0c)
|

```

清除窗口 打开文件 发送文件

端口号 COM61 USB-SERIAL CH340 ☐ HEX显示 ☒ 保存数据 ☐ 接收数据至

☒ 关闭串口 ☒ 更多串口设置 ☒ 加时间戳和分包显示 超时时间: 20

☐ RTS ☒ DTR 波特率: 115200 CCCC00

Figure 4-2: Serial port operation log

3. Send hexadecimal command `CCCC05` via serial port to write the customized AES key, then send `CCCC01` to read back the key and verify the consistency of written and read values;
4. Send hexadecimal command `CCCC04` via serial port to enable the AES hardware function, then send `CCCC00` to read the

enable state; if the return value is 1, the initialization is successful;

- Power on the device again, complete the permanent AES function solidification, and the chip will no longer support plaintext firmware operation.

5 Application Examples

5.1 Example 1

Objective: Realize permanent full-Flash AES encryption based on eFuse, and complete encrypted firmware downloading and normal startup.

Configuration: Complete eFuse AES key burning and function enabling in advance; prepare dedicated AES encrypted FLM file or matched key configuration file.

Operation Flow (Keil+FLM Download Method):

- Unzip `CBU5000V210\_FLM.7z` in the SDK `.\Tools\FLM` directory;
- Open the FLM project, modify the encryption macro configuration, compile and generate the encrypted .FLM file;

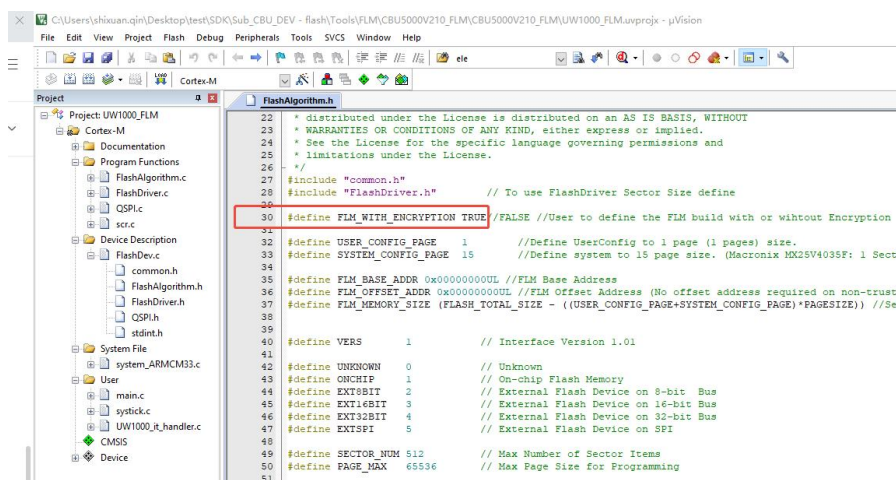


Figure 5-1: FLM project configuration interface

- Open the target firmware project, select the newly generated encrypted FLM file in Keil configuration;
- Compile and download the firmware to Flash;
- Power on the device again and observe the serial port output.

Expected Result: The serial port outputs normal logs, which indicates that the encrypted firmware is successfully downloaded, and the chip hardware AES module automatically decrypts the firmware data and runs the program normally.

5.2 Example 2

Objective: Generate plaintext bin file via Keil compilation, encrypt the bin file through offline encryption tool, and complete encrypted firmware download with SF100 tool.

Configuration: Complete eFuse AES key burning and permanent AES enabling in advance; prepare matching key.key file and Encryption.exe offline encryption tool; ensure the AES key and seed in key.key are completely consistent with the values burned into eFuse.

Operation Flow (Keil Compile + Offline Encryption + SF100 Download Method):

1. Open the SDK `peripheral\_flash\_encryption` example project, select the unencrypted FLM file in Keil configuration.
2. Modify the **auto.bat** script file in the project Toolchains folder to ensure the tool path configuration is correct.

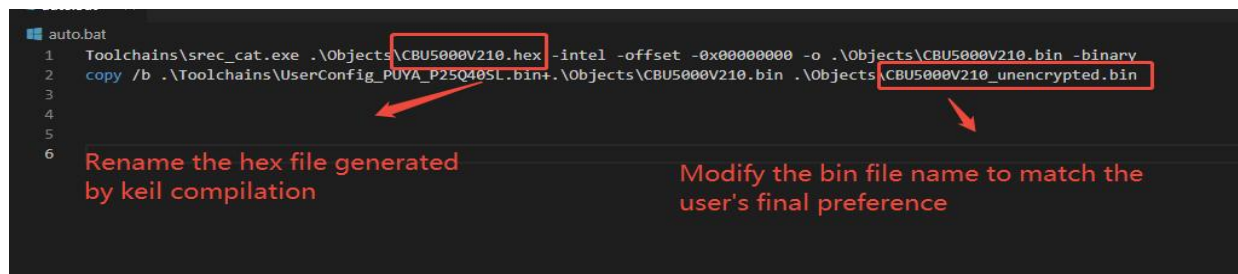


Figure 5-2: Modification content of auto.bat batch script for plaintext bin generation

3. Open Keil `Option for Target` interface, select the User tab, only check `Run #1` in the After Build/Rebuild option.

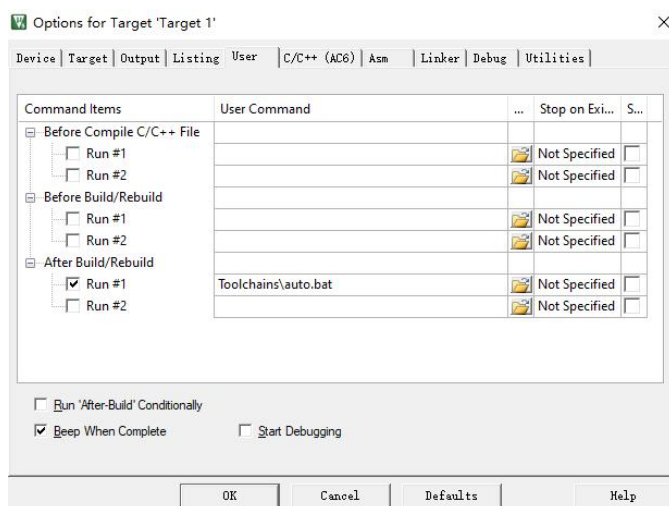


Figure 5-3: Keil After-build run settings

4. Complete the above configuration and compile the project. An unencrypted original bin file will be automatically generated in the project Object folder.
5. Enter the path `...\peripheral\_flash\_encryption\Keil\Toolchains` in the project, modify the key and seed in the key.key file to be consistent with the 128-bit AES key and seed burned by eFuse, and strictly follow the key format requirements.

EFUSE	W4	W5	W6	W7	W8
FW (AES Key)	-	0x01234567	0x89ABCDEF	0x01234567	0x89ABCDEF
FW (Seed)	0x12345678	-	-	-	-
Key.Key (AES Key)	-	89AB-CDEF	0123-04567	89AB-CDEF	0123-4567
Key.Key (Seed)	1234-5678	-	-	-	-

Figure 5-4: Key and Seed mapping table

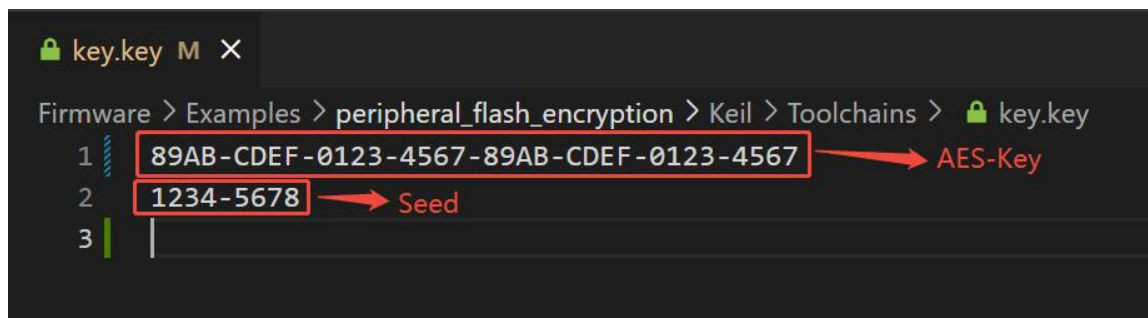


Figure 5-5: key.key file format example

6. Copy the unencrypted original bin file generated in the Object directory to the Toolchains directory (where Encryption.exe is located).
7. Perform offline encryption: Directly drag the target bin file onto 'Encryption.exe', or run the following command in CMD: 'Encryption.exe xxx.bin'. The encrypted output file 'xxx.encrypted.bin' will be automatically generated after execution.

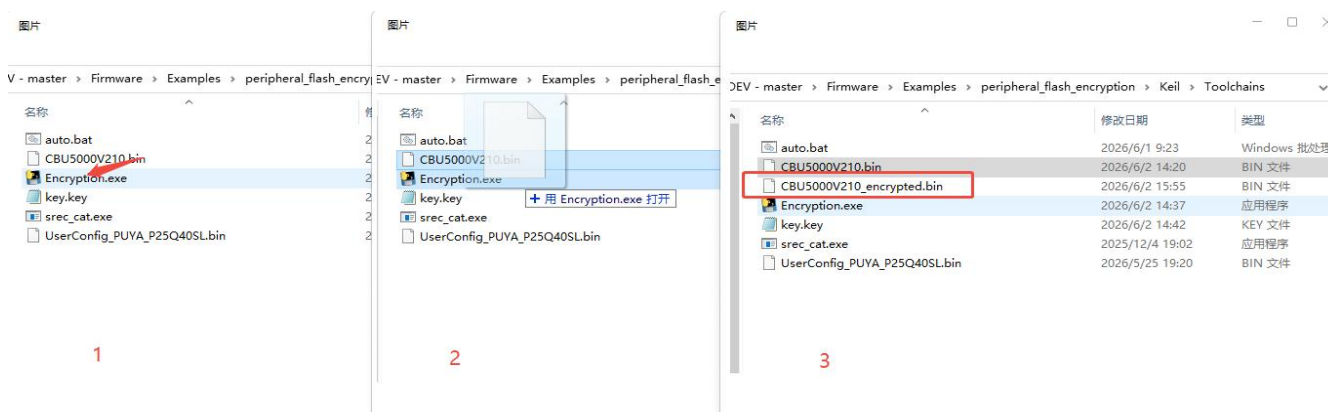


Figure 5-6: Offline encryption tool operation method

8. Connect the target Flash chip via SF100, select the 'xxx.encrypted.bin' file generated by the encryption tool and download it.

Expected Result: After downloading and restarting the device, normal serial port output confirms that the encrypted firmware is downloaded successfully and decrypted correctly by the hardware AES module, and the program runs stably.

5.3 Example 3

Objective: Verify the effectiveness of register-based selective Flash AES partial encryption function.

Configuration: Use SDK peripheral_flash_encryption example project, switch AES enable or disable status by calling dedicated API functions for comparative test.

Operation Flow

1. Run Flash read/write test without invoking AES-related APIs, compile and download the program, confirm serial port outputs normal Flash access log under plaintext mode;
2. Call `cb_flash_encryption_enable()` before executing Flash erase or write operations;
3. Invoke `cb_flash_encryption_disable()` after all data access is finished;
4. Recompile and download the firmware. The serial output works normally, and all Flash written data is stored as ciphertext;
5. Remove AES enable/disable API calls and comment out Flash erase-write logic in `app_flash.c`, recompile and download, then check data verification result after power-up;

6. Restore the two AES switch APIs, recompile and verify Flash data read status again.

Expected Result: After removing AES control APIs, Flash data check fails since stored ciphertext cannot be decrypted under plaintext mode; verification returns to normal after recovering enable/disable APIs. This test confirms partial AES encryption works correctly, and written Flash data is automatically hardware-encrypted.

6 Firmware APIs

Two dedicated APIs are provided to dynamically switch Flash partial AES encryption function, valid for register-controlled optional encryption mode only.

1. void cb_flash_encryption_enable(void)

Function: Enable hardware AES auto-encryption/decryption for subsequent Flash write/read operations.

2. void cb_flash_encryption_disable(void)

Function: Disable hardware AES, Flash operates in plaintext read/write mode afterwards.

Usage Example:

```
cb_flash_init();  
cb_flash_encryption_enable(); // Turn on AES before Flash operation  
cb_flash_erase_sector(...);  
cb_flash_program_by_addr(...);  
cb_flash_read_by_addr(...);  
cb_flash_encryption_disable(); // Turn off AES after data access  
.....
```

7 Note

1. eFuse AES encryption is one-time irreversible: After burning the key and enabling the AES function, it cannot be modified or closed permanently, and the chip can only run encrypted firmware for life;
2. Key matching restriction: The encrypted firmware must use the exact same 128-bit AES key and seed as the eFuse burned key, and mismatched key or seed will lead to firmware decryption failure and startup crash;
3. FLM file matching restriction: Encrypted downloading via FLM requires specially compiled AES encrypted FLM file; ordinary plaintext FLM files cannot complete encrypted downloading;
4. Partial encryption validity restriction: Register-based partial AES encryption stores ciphertext data in Flash after being written in enabled state. Ciphertext data can still be correctly decrypted in subsequent power-on sessions as long as the AES module is re-enabled with the same key, even if the module was disabled between operations. The encryption function does not affect or encrypt existing plaintext data stored in Flash before AES enabling.
5. The bin file encrypted by Encryption.exe must match the key and seed configured in key.key (which must be identical to the values burned into eFuse), otherwise the firmware cannot be decrypted normally.

8 Revision History

Version No	Description	Date
1.0	Document Creation	03 June 2026
1.1	Change Formatting	03 September 2026

9 Disclaimers

CHIPSBANK INTENDS FOR THE CONTENT CONTAINED IN THE DOCUMENT TO BE ACCURATE AND RELIABLE. THIS CONTENT MAY, HOWEVER, CONTAIN TECHNICAL INACCURACIES, TYPOGRAPHICAL ERRORS OR OTHER MISTAKES. CHIPSBANK MAY MAKE CORRECTIONS OR OTHER CHANGES TO THIS CONTENT AT ANY TIME. CHIPSBANK AND ITS SUPPLIERS RESERVE THE RIGHT TO MAKE CORRECTIONS, MODIFICATIONS, ENHANCEMENTS, IMPROVEMENTS AND OTHER CHANGES TO ITS PRODUCTS, PROGRAMS AND SERVICES AT ANY TIME OR TO DISCONTINUE ANY PRODUCTS, PROGRAMS, OR SERVICES WITHOUT NOTICE.

THE CONTENT IN THIS DOCUMENT IS PROVIDED "AS IS". CHIPSBANK AND ITS RESPECTIVE SUPPLIERS MAKE NO REPRESENTATIONS ABOUT THE SUITABILITY OF THIS CONTENT FOR ANY PURPOSE AND DISCLAIM ALL WARRANTIES AND CONDITIONS WITH REGARD TO THIS CONTENT, INCLUDING BUT NOT LIMITED TO, ALL IMPLIED WARRANTIES AND CONDITIONS OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT OF ANY THIRD PARTY INTELLECTUAL PROPERTY RIGHT.

CHIPSBANK DOES NOT WARRANT OR REPRESENT THAT ANY LICENSE, EITHER EXPRESS OR IMPLIED, IS GRANTED UNDER ANY PATENT RIGHT, COPYRIGHT, MASK WORK RIGHT, OR OTHER INTELLECTUAL PROPERTY RIGHT OF CHIPSBANK COVERING OR RELATING TO THIS CONTENT OR ANY COMBINATION, MACHINE, OR PROCESS TO WHICH THIS CONTENT RELATE OR WITH WHICH THIS CONTENT MAY BE USED.

USE OF THE INFORMATION IN THIS DOCUMENT MAY REQUIRE A LICENSE FROM A THIRD PARTY UNDER THE PATENTS OR OTHER INTELLECTUAL PROPERTY OF THAT THIRD PARTY, OR A LICENSE FROM CHIPSBANK UNDER THE PATENTS OR OTHER INTELLECTUAL PROPERTY OF CHIPSBANK.

INFORMATION IN THIS DOCUMENT IS PROVIDED SOLELY TO ENABLE SYSTEM AND SOFTWARE IMPLEMENTERS TO USE CHIPSBANK PRODUCTS. THERE ARE NO EXPRESS OR IMPLIED COPYRIGHT LICENSES GRANTED HEREUNDER TO DESIGN OR FABRICATE ANY INTEGRATED CIRCUITS OR INTEGRATED CIRCUITS BASED ON THE INFORMATION IN THIS DOCUMENT. CHIPSBANK RESERVES THE RIGHT TO MAKE CHANGES WITHOUT FURTHER NOTICE TO ANY PRODUCTS HEREIN. CHIPSBANK MAKES NO WARRANTY, REPRESENTATION OR GUARANTEE REGARDING THE SUITABILITY OF ITS PRODUCTS FOR ANY PARTICULAR PURPOSE, NOR DOES CHIPSBANK ASSUME ANY LIABILITY ARISING OUT OF THE APPLICATION OR USE OF ANY PRODUCT OR CIRCUIT, AND SPECIFICALLY DISCLAIMS ANY AND ALL LIABILITY, INCLUDING WITHOUT LIMITATION CONSEQUENTIAL OR INCIDENTAL DAMAGES. "TYPICAL"

PARAMETERS WHICH MAY BE PROVIDED IN CHIPSBANK DATA SHEETS AND/OR SPECIFICATIONS CAN AND DO VARY IN DIFFERENT APPLICATIONS AND ACTUAL PERFORMANCE MAY VARY OVER TIME. ALL OPERATING PARAMETERS, INCLUDING "TYPICALS" MUST BE VALIDATED FOR EACH CUSTOMER APPLICATION BY CUSTOMER'S TECHNICAL EXPERTS. CHIPSBANK DOES NOT CONVEY ANY LICENSE UNDER NEITHER ITS PATENT RIGHTS NOR THE RIGHTS OF OTHERS. CHIPSBANK PRODUCTS ARE NOT DESIGNED, INTENDED, OR AUTHORIZED FOR USE AS COMPONENTS IN SYSTEMS INTENDED FOR SURGICAL IMPLANT INTO THE BODY, OR OTHER APPLICATIONS INTENDED TO SUPPORT OR SUSTAIN LIFE, OR FOR ANY OTHER APPLICATION IN WHICH THE FAILURE OF THE CHIPSBANK PRODUCT COULD CREATE A SITUATION WHERE PERSONAL INJURY OR DEATH MAY OCCUR. SHOULD BUYER PURCHASE OR USE CHIPSBANK PRODUCTS FOR ANY SUCH UNINTENDED OR UNAUTHORIZED APPLICATION, BUYER SHALL INDEMNIFY AND HOLD CHIPSBANK AND ITS OFFICERS, EMPLOYEES, SUBSIDIARIES, AFFILIATES, AND DISTRIBUTORS HARMLESS AGAINST ALL CLAIMS, COSTS, DAMAGES, AND EXPENSES, AND REASONABLE ATTORNEY FEES ARISING OUT OF, DIRECTLY OR INDIRECTLY, ANY CLAIM OF PERSONAL INJURY OR DEATH ASSOCIATED WITH SUCH UNINTENDED OR UNAUTHORIZED USE, EVEN IF SUCH CLAIM ALLEGES THAT CHIPSBANK WAS NEGLIGENT REGARDING THE DESIGN OR MANUFACTURE OF THE PART.